

Backend Core + Frontend

Role

You own the project skeleton. Every other person's component plugs into what you build. You are responsible for the database, all session lifecycle logic, the QA verdict, trust score updates, the warning relay, and the React frontend users interact with.

Data Flow

```
Person 4 (warnings)      → POST /warnings/log → WebSocket broadcast → Frontend
Person 4 (drift summary) → POST /session/{id}/drift-summary → triggers verdict
Frontend (mic audio)     → streams to Person 2's server (port 8001)
Frontend                 → polls GET /session/{id}/status
```

What You Build

1. Project Setup

- FastAPI backend on **port 8000**
 - PostgreSQL database with migrations
 - React frontend (3 screens)
 - `.env` for DB URL, secret key, port
-

2. Database Schema

users

Column	Type	Notes
user_id	serial PK	
name	varchar	
trust_score	float	default 0.30

Seed two hardcoded users on first run: Alice (user_id 1), Bob (user_id 2).

barters

Column	Type	Notes
barter_id	serial PK	
user_a_id	int FK → users	
user_b_id	int FK → users	
skill_a	varchar	what Alice teaches
skill_b	varchar	what Bob teaches
status	varchar	proposed → active → completed
created_at	timestamp	

session_contracts

Column	Type	Notes
contract_id	serial PK	
barter_id	int FK	
topic	varchar	
scope	varchar	beginner/intermediate
allowed_concepts	text	comma-separated
disallowed_concepts	text	comma-separated
agreed_duration_minutes	int	
started_at	timestamp	nullable
completed_at	timestamp	nullable

warnings

Column	Type	Notes
warning_id	serial PK	
barter_id	int FK	
severity	varchar	mild / strong / severe
reason	text	
window_ids	text	comma-separated
timestamp	timestamp	

qa_verdicts

Column	Type	Notes
verdict_id	serial PK	
barter_id	int FK	
verdict	varchar	SUCCESSFUL / PARTIAL / DISPUTE
duration_check	boolean	
confirmation_check	boolean	
drift_summary	text	JSON string from Person 4
created_at	timestamp	

trust_scores

Column	Type	Notes
trust_id	serial PK	
user_id	int FK, unique	
score	float	
successful_barters	int	default 0
disputed_barters	int	default 0
update_reason	varchar	
updated_at	timestamp	

Seed initial trust records for Alice and Bob (score = 0.30).

confirmations

Column	Type	Notes
confirmation_id	serial PK	
barter_id	int FK	
user_id	int FK	
confirmed_at	timestamp	

3. Session Lifecycle Endpoints

POST `/session/create`

Creates barter + session contract. Also calls Person 3's contract registration and Person 4's session init.

Request:

```
{
  "skill_a": "python basics",
  "skill_b": "graphic design",
  "topic": "Python Basics",
  "scope": "beginner",
  "allowed_concepts": ["variables", "loops", "functions", "syntax"],
  "disallowed_concepts": ["decorators", "async", "metaclasses"],
  "agreed_duration_minutes": 45
}
```

Response:

```
{
  "barter_id": 1,
  "contract_id": 1,
}
```

```
"status": "proposed"
}
```

POST /session/{barter_id}/start

Records start time. Sets barter status to `active`.

Response:

```
{
  "barter_id": 1,
  "started_at": "2026-03-12T10:00:00",
  "status": "active"
}
```

POST /session/{barter_id}/confirm

Called by each user when they click "Session Complete." Once both users confirm, marks barter `completed` and triggers verdict generation.

Request:

```
{ "user_id": 1 }
```

Response:

```
{
  "barter_id": 1,
  "confirmed_by": [1],
  "both_confirmed": false
}
```

GET /session/{barter_id}/status

Polled by frontend to check state.

Response:

```
{
  "barter_id": 1,
  "status": "active",
  "elapsed_minutes": 23,
  "agreed_duration_minutes": 45,
  "can_complete": false,
  "both_confirmed": false
}
```

POST `/session/{barter_id}/terminate`

Called by Person 4 on severe drift. Force-closes session and sets verdict to DISPUTE.

Request:

```
{ "reason": "Severe and persistent topic drift detected" }
```

Response:

```
{ "barter_id": 1, "status": "terminated" }
```

4. Warning Relay Endpoint

POST `/warnings/log`

Called by Person 4. Saves warning to DB, then broadcasts it to the frontend via WebSocket.

Request:

```
{
  "barter_id": 1,
```

```
"severity": "mild",
"reason": "Session appears to be drifting off topic",
"window_ids": "3,4",
"timestamp": "2026-03-12T10:23:45"
}
```

Response:

```
{ "warning_id": 7 }
```

5. WebSocket (Warnings → Frontend)

```
ws://localhost:8000/ws/warnings/{barter_id}
```

Frontend connects on session start. When a warning is logged via `POST /warnings/log`, broadcast the payload to all connected clients on that barter's channel.

6. Drift Summary Receiver

`POST /session/{barter_id}/drift-summary`

Called by Person 4 at end of session. Store as JSON string in `qa_verdicts.drift_summary`, then trigger verdict generation.

Request:

```
{
  "barter_id": 1,
  "total_windows": 12,
  "incorrect_windows": 3,
  "percent_incorrect": 25.0,
  "max_consecutive_incorrect": 2,
  "warning_count": 1,
  "warnings": [],
}
```

```
"terminated_early": false
}
```

Response:

```
{ "status": "received" }
```

7. QA Verdict

POST `/verdict/{barter_id}/generate`

Auto-triggered after drift summary is received. Applies 3 rule checks.

Rules:

- `duration_check` = actual duration $\geq$ 80% of agreed duration
- `confirmation_check` = both users confirmed
- SUCCESSFUL $\rightarrow$ both checks pass
- PARTIAL $\rightarrow$ one check passes
- DISPUTE $\rightarrow$ neither check passes, OR session was terminated early

Response:

```
{
  "verdict": "SUCCESSFUL",
  "duration_check": true,
  "confirmation_check": true
}
```

8. Trust Update

POST `/trust/{barter_id}/update`

Auto-triggered right after verdict. Updates both users' trust scores using the formula below.

Formula:

```
quality_adjusted = (qa_score + 0.8) / 2
  where qa_score: SUCCESSFUL=1.0, PARTIAL=0.5, DISPUTE=0.0
  satisfaction_rating hardcoded to 4/5 for POC

new_trust = (previous_trust × 0.3) + (quality_adjusted × 0.7) ← first
session
new_trust = clamp(new_trust, 0.0, 1.0)
```

Response:

```
{
  "user_1_trust": { "before": 0.30, "after": 0.65 },
  "user_2_trust": { "before": 0.30, "after": 0.65 }
}
```

9. Frontend (React – 3 Screens)

Screen 1: Session Setup

- Fields: Topic, Scope (dropdown), Allowed Concepts (comma-separated), Disallowed Concepts (comma-separated), Duration (minutes)
- On submit → `POST /session/create` → navigate to Screen 2

Screen 2: Live Session

- Start button → `POST /session/{barter_id}/start`
- Mic capture via browser `MediaRecorder` API → streams audio chunks to `ws://localhost:8001/audio/{barter_id}/{user_id}` (Person 2's server)
- Connect to `ws://localhost:8000/ws/warnings/{barter_id}` → display warning banners as they arrive

- Yellow = mild, Orange = strong, Red = severe
- Session timer showing elapsed vs agreed duration
- "Mark Complete" button → `POST /session/{barter_id}/confirm`
- Auto-navigate to Screen 3 when `both_confirmed: true`

Screen 3: Post Session

- Fetch verdict from `GET /verdict/{barter_id}`
- Show: verdict type, duration check result, confirmation check result
- Show drift summary: % off-topic, number of warnings
- Show trust score before and after for both users

Interface Contracts

You Call (outbound)

Target	When	Endpoint
Person 3	On session create	<code>POST http://localhost:8002/session/{barter_id}/contract</code>
Person 4	On session start	<code>POST http://localhost:8003/session/{barter_id}/init</code>
Person 2	On session complete	<code>POST http://localhost:8001/session/{barter_id}/end</code>

You Expose (inbound)

Caller	Endpoint
Person 4	<code>POST /warnings/log</code>
Person 4	<code>POST /session/{barter_id}/drift-summary</code>
Person 4	<code>POST /session/{barter_id}/terminate</code>
Frontend	<code>ws://localhost:8000/ws/warnings/{barter_id}</code>

What You Do NOT Build

- Whisper STT or audio processing → Person 2
- Sentence-BERT embeddings or similarity scoring → Person 3
- Sliding window logic → Person 3
- Warning severity decisions → Person 4
- The WebSocket server that receives audio from the browser → Person 2 (port 8001)

Libraries

```
fastapi, uvicorn, sqlalchemy, alembic, psycpg2-binary, python-dotenv,  
websockets, httpx  
react (frontend)
```

Definition of Done

- ☐ DB schema created and all tables exist
- ☐ All session lifecycle endpoints return correct responses
- ☐ Warning relay saves to DB and broadcasts to WebSocket clients
- ☐ Drift summary stores correctly as JSON
- ☐ QA verdict generates correctly for all 3 verdict types
- ☐ Trust scores update after verdict
- ☐ Frontend Screen 1: session created successfully
- ☐ Frontend Screen 2: mic captures and streams audio, warning banners display
- ☐ Frontend Screen 3: verdict and trust scores displayed correctly